

UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Exercice 2: Sorting

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors
TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be

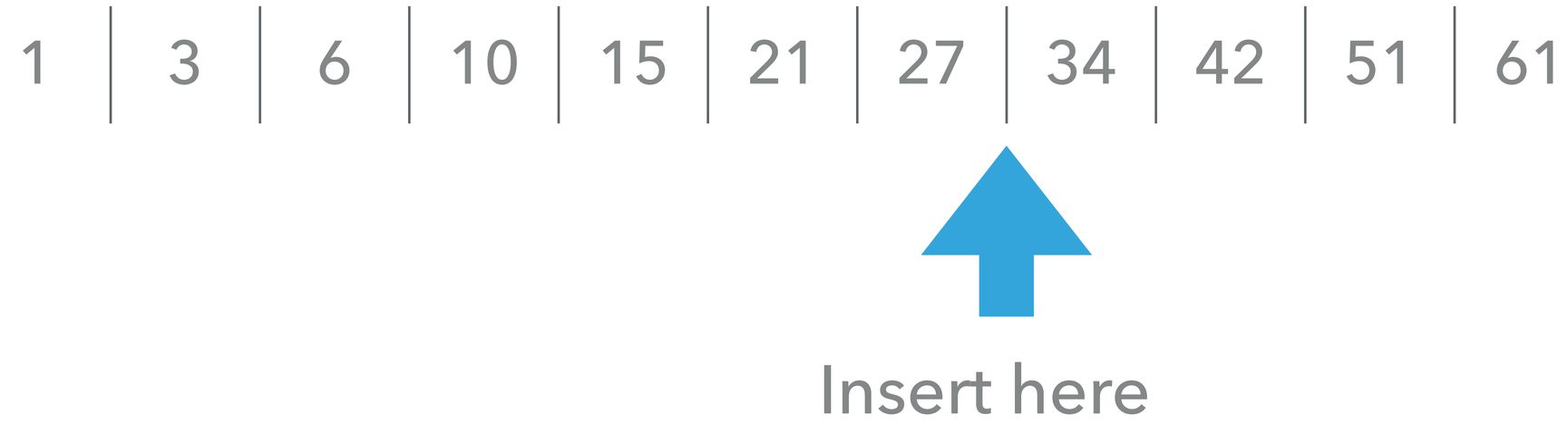
Question 2.1.1: Insert element in a sorted array

1	3	6	10	15	21	27	34	42	51

Where to insert 30?

Question 2.1.1: Insert element in a sorted array

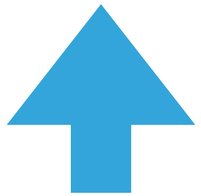
From left to right:



Question 2.1.1: Insert element in a sorted array

Binary Search :

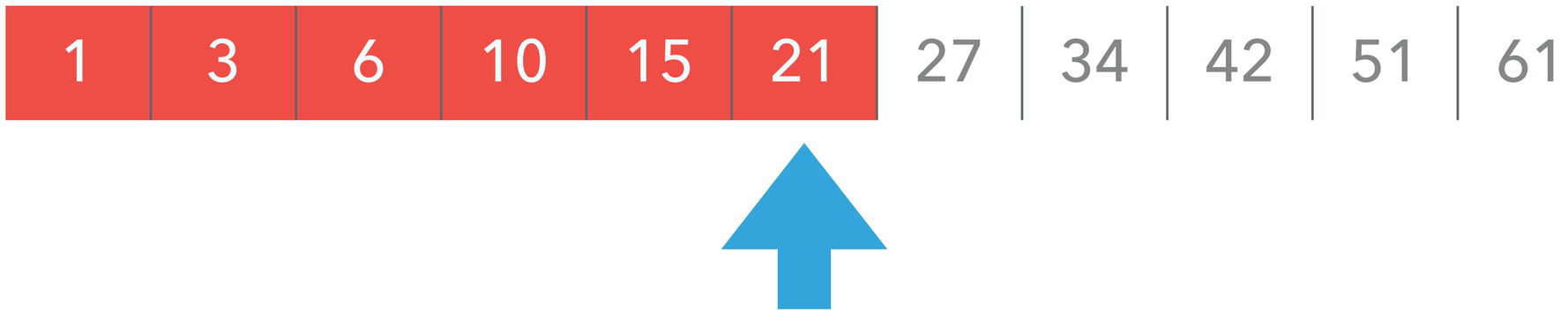
1	3	6	10	15	21	27	34	42	51	61



21 < 30: on the right !

Question 2.1.1: Insert element in a sorted array

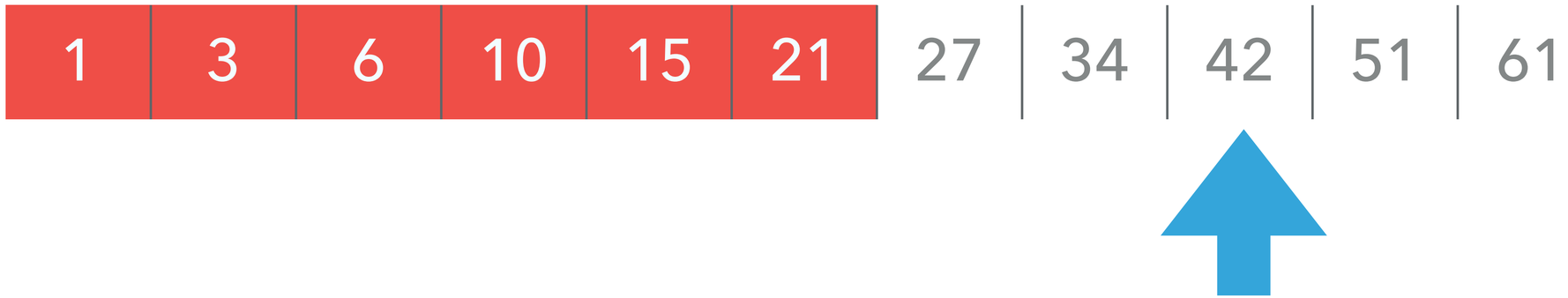
Binary Search :



21 < 30: on the right !

Question 2.1.1: Insert element in a sorted array

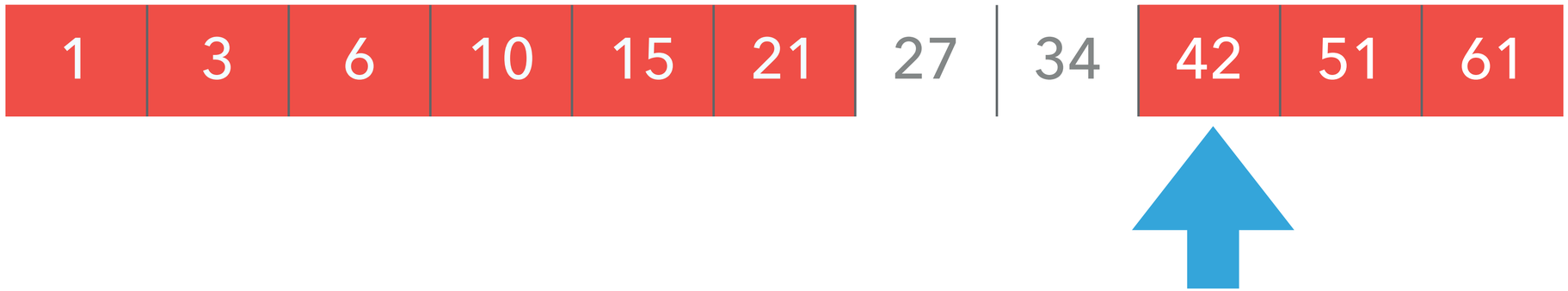
Binary Search :



42 > 30: on the left !

Question 2.1.1: Insert element in a sorted array

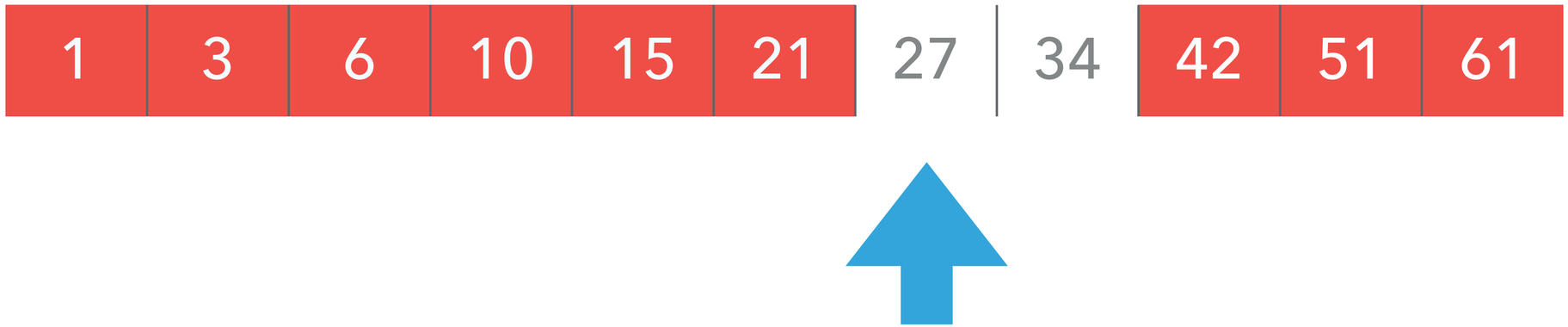
Binary Search :



42 > 30: on the left !

Question 2.1.1: Insert element in a sorted array

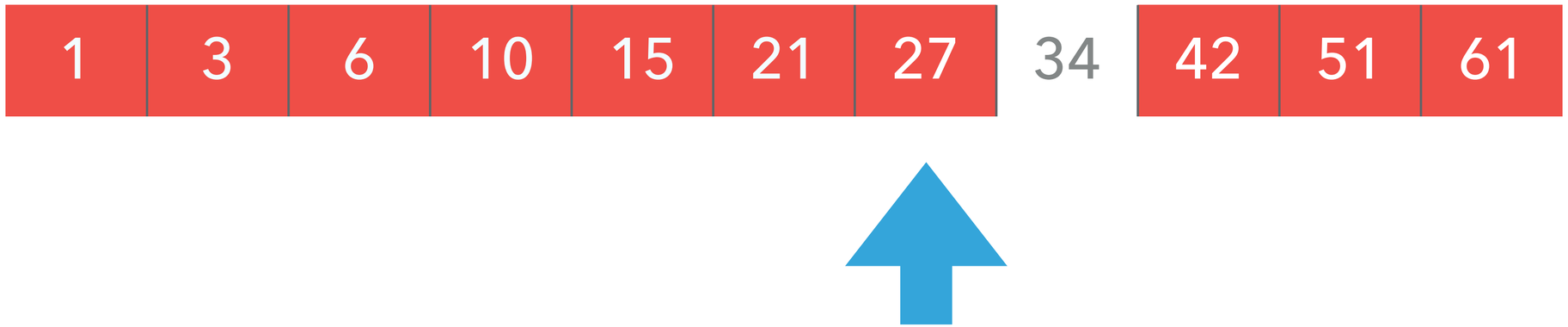
Binary Search :



27 < 30: on the right !

Question 2.1.1: Insert element in a sorted array

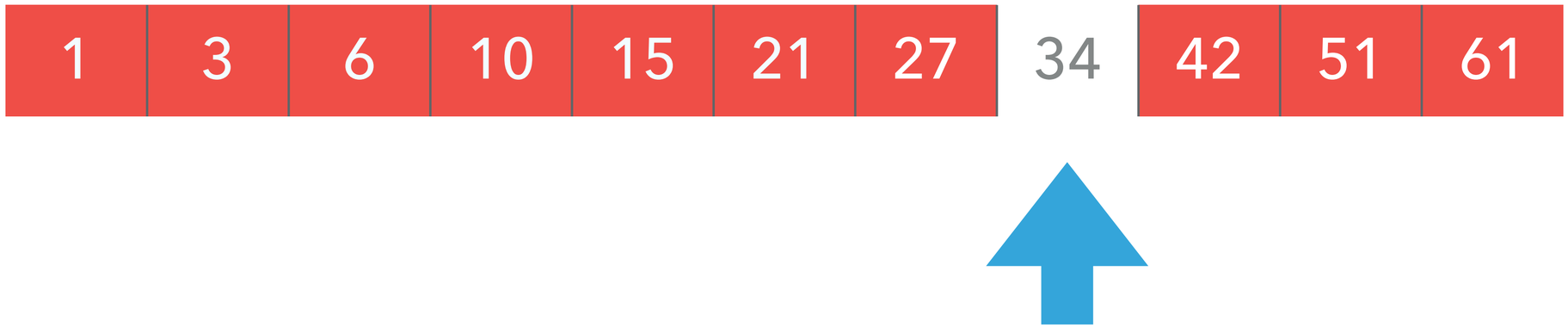
Binary Search :



27 < 30: on the right !

Question 2.1.1: Insert element in a sorted array

Binary Search :



34 > 30: on the left !

Question 2.1.1: Insert element in a sorted array

Binary Search :



Question 2.1.1: Insert element in a sorted array

Binary Search :

What's the time complexity ?

$$f(n) = 1 + f(\frac{n}{2})$$
$$f(1) = 0$$

$$f(n) = \log_2 n$$

Question 2.1.1: Insert element in a sorted array

Can we do something faster than that ?

Yes, if we have information on the data distribution in the array !

Question 2.1.2: Maximizing customer satisfaction

We consider the very general problem where we have n jobs to perform for clients and each job j takes t_j seconds to complete. Only one job can be performed at a time.

The goal is to complete all jobs while maximizing customer satisfaction. Maximizing customer satisfaction means building a schedule that *minimizes the average job completion time*.

Example: if we have four jobs that take respectively 5, 8, 3, 1 seconds to finish then the order 1,2,3,4 takes an average completion time of (5 + 13 + 16 + 17) /4=12.75

Question 2.1.2: Maximizing customer satisfaction

Solution: sort the jobs in increasing processing time t_j

Example: if we have four jobs that take respectively 5, 8, 3, 1 seconds to finish then the optimal order 4,3,1,2 takes an average completion time of (1 + 4 + 9 + 17) /4 =7.75

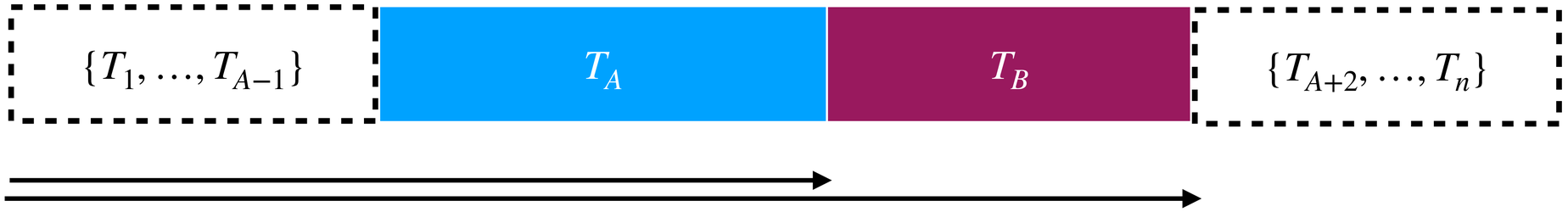
Question 2.1.2: Maximizing customer satisfaction

Ojective:

$$\min \frac{\sum_{i=1}^{n} (\sum_{j=1}^{i} t_j)}{n}$$

Let's prove it by contradiction.

Assume that we have a job A which is done just before B but takes more time than B: $t_A > t_B$



Supposed optimal

$$T_1 = (\sum_{i=1}^{A-1} t_i) + t_A + (\sum_{i=1}^{A-1} t_i) + t_A + t_B$$

Completion times if A and B are inverted

$$T_2 = (\sum_{i=1}^{A-1} t_i) + t_B + (\sum_{i=1}^{A-1} t_i) + t_B + t_A$$

Question 2.1.2: Maximizing customer satisfaction

$$T_1 = (\sum_{i=1}^{A-1} t_i) + t_A + (\sum_{i=1}^{A-1} t_i) + t_A + t_B \qquad T_2 = (\sum_{i=1}^{A-1} t_i) + t_B + (\sum_{i=1}^{A-1} t_i) + t_B + t_A$$

$$T_1 - T_2 = (\sum_{i=1}^{A-1} t_i) + t_A + (\sum_{i=1}^{A-1} t_i) + t_A + t_B - ((\sum_{i=1}^{A-1} t_i) + t_B + (\sum_{i=1}^{A-1} t_i) + t_B + t_A)$$

$$T_1 - T_2 = t_A + t_A + t_B - (t_B + t_B + t_A) = t_A - t_B > 0$$

By inverting A and B, we reduce the average time, we have a contradiction.

Since this must hold for every pair of job in the ordering, the jobs must be ordered by duration time.

Question 2.1.4 Cards sorter

How would you sort increasingly a pile of cards with the restriction that the only permitted operations are:

1. compare the first two cards,

1. exchange the first two cards,

3. move the first card to the back of the pile?

Question 2.1.4 Cards sorter

How would you sort increasingly a pile of cards with the restriction that the only permitted operations are:

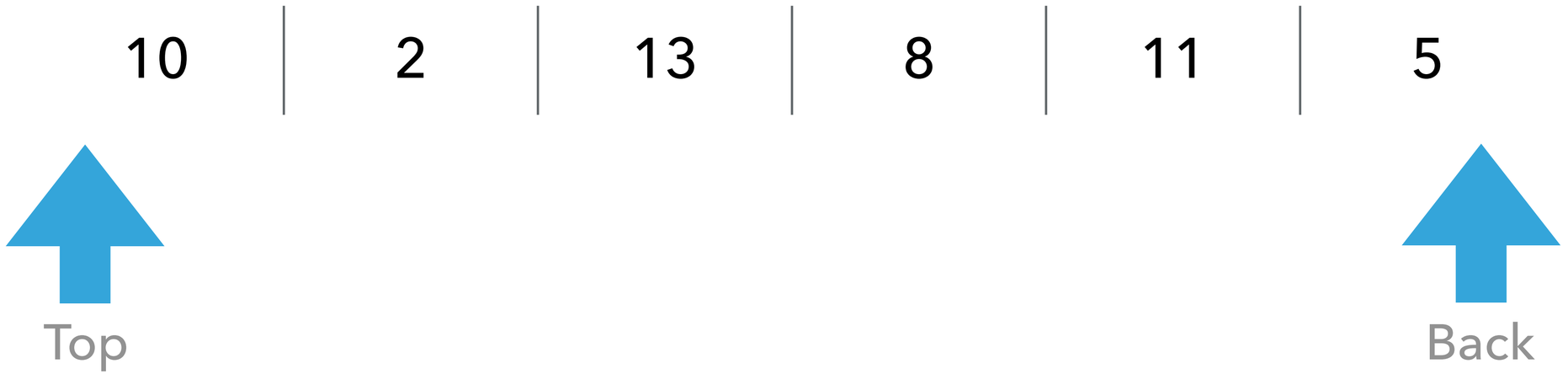
1. compare the first two cards,

1. exchange the first two cards,

3. move the first card to the back of the pile?

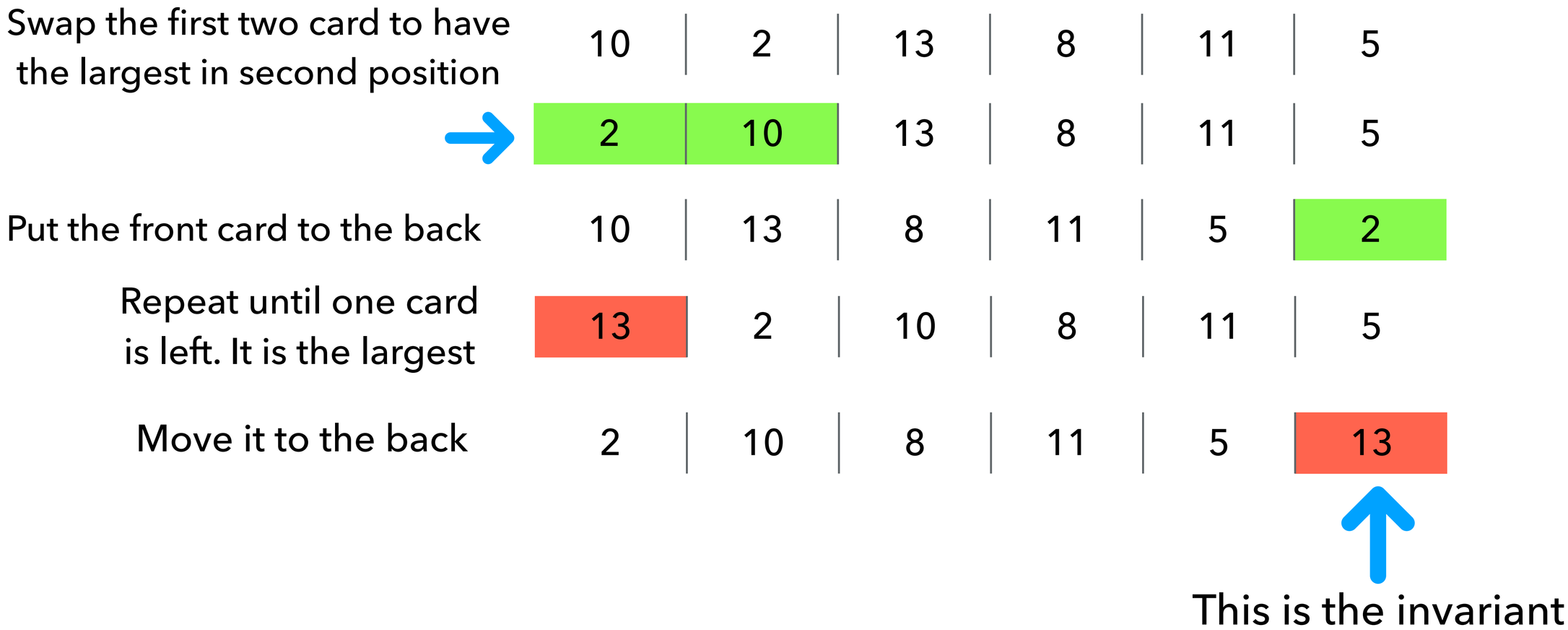
Idea : Try to maintain the **invariant** that the last i elements of the pile are sorted and those are the ith biggest ones.
After n iteration, the last n elements are sorted !

Question 2.1.4 Cards sorter



Question 2.1.4 Cards sorter

First iteration, find the largest element and put it at the end !



Question 2.1.4 Cards sorter

What about the next iterations ?

2	10	8	11	5	13

Same process. Find the largest in the n-i first element, put it at the end !

13	11	1	8	10	5

1	8	10	5	11	13

Question 2.1.4 Cards sorter

```
for (int i = 0; i < n; i++) {
    // Invariant: the i last elements are sorted and are the largest ones
    for (int k = 0; k < n; k++) {
        if (k <= n - 1 - i) {
            // put the smallest of the two top card on top
        }
        // move the top card at the end
    }
}
```

Question 2.1.5 Sorting a simply linked list

How to sort a doubly linked list (which therefore does not allow access to a position by its index) efficiently? How complex is your algorithm?

Question 2.1.5 Sorting a double simply list

How to sort a doubly linked list (which therefore does not allow access to a position by its index) efficiently? How complex is your algorithm?

Can we take ideas from know sorting algorithms ?

Question 2.1.5 Sorting a simply linked list

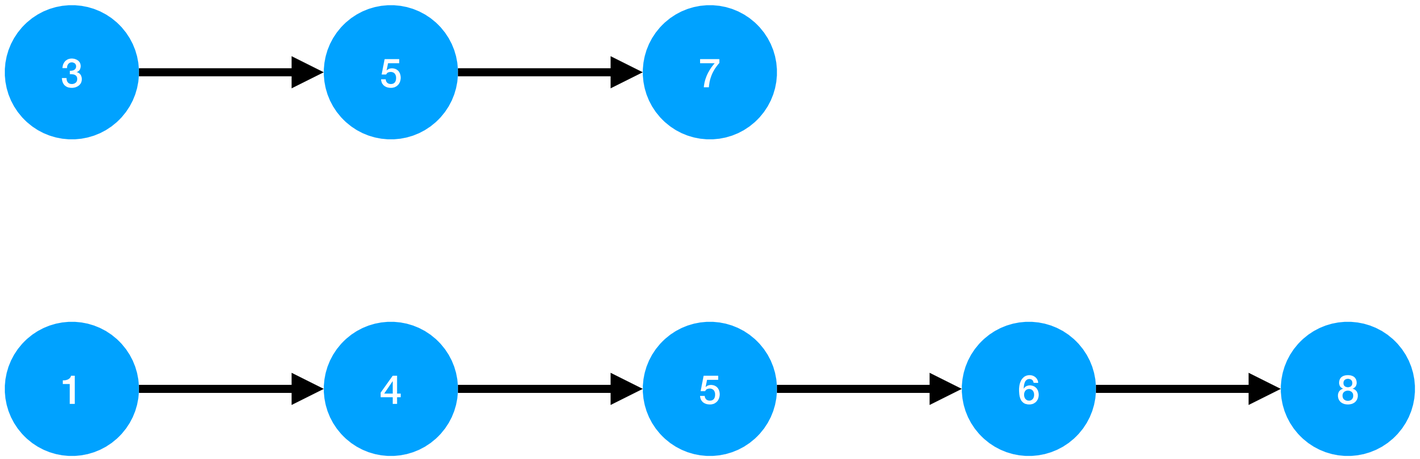
Merging two linked-list is similar to merging arrays in the MergeSort algorithm !
The "merge" operation can also be done in O(n+m) for linked list

```
public static void merge(Comparable[] a, int lo, int mid, int hi)
{  // Merge a[lo..mid] with a[mid+1..hi].
   int i = lo, j = mid+1;

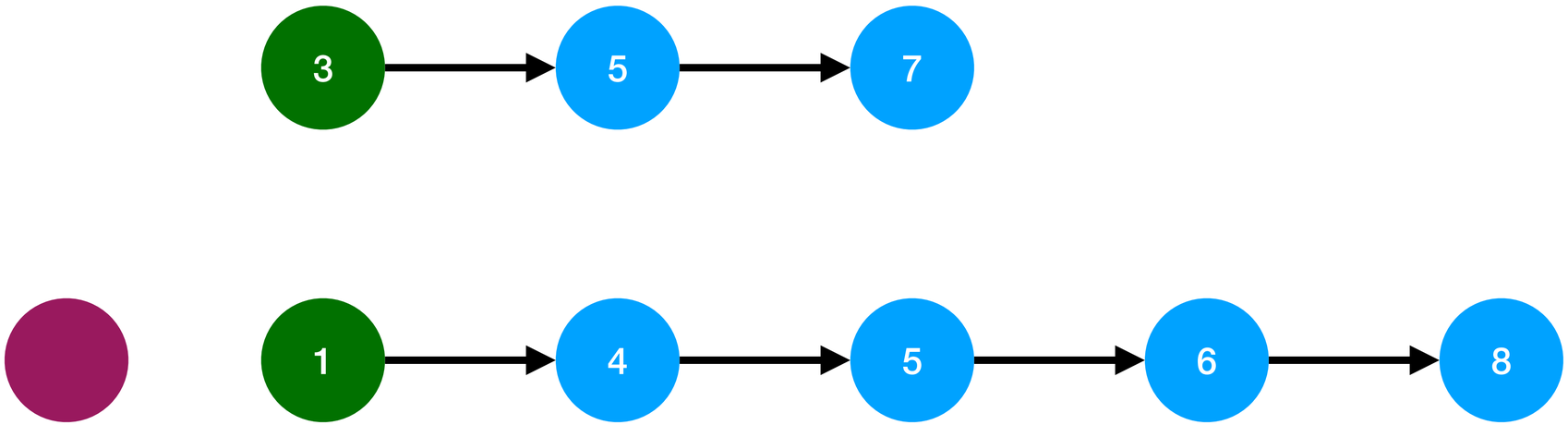
   for (int k = lo; k <= hi; k++)  // Copy a[lo..hi] to aux[lo..hi].
      aux[k] = a[k];

   for (int k = lo; k <= hi; k++)  // Merge back to a[lo..hi].
      if       (i > mid)                  a[k] = aux[j++];
      else if (j > hi )                   a[k] = aux[i++];
      else if (less(aux[j], aux[i])) a[k] = aux[j++];
      else                                a[k] = aux[i++];
}
```

Merge of two sorted LinkedLists

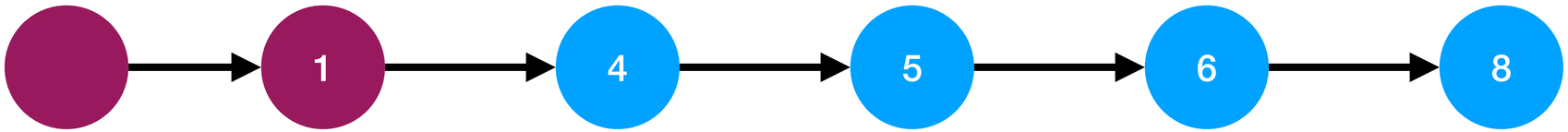


Merge of two sorted LinkedLists



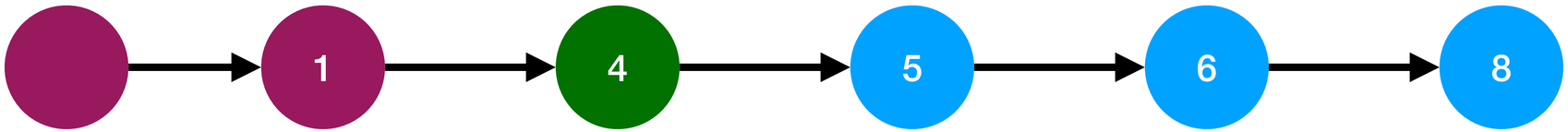
Merge of two sorted LinkedLists



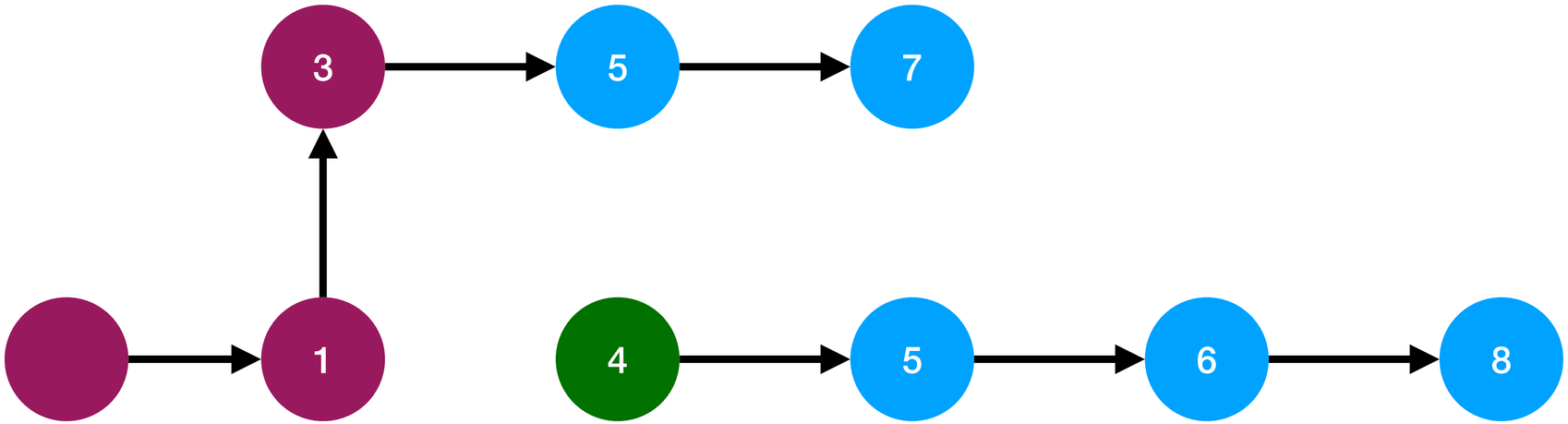


Merge of two sorted LinkedLists

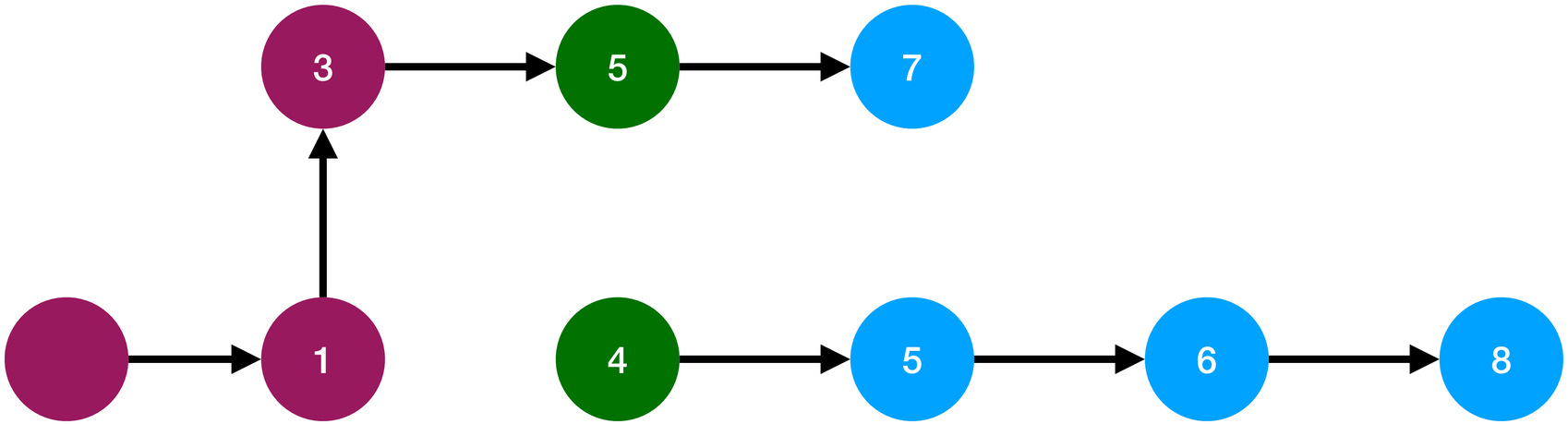




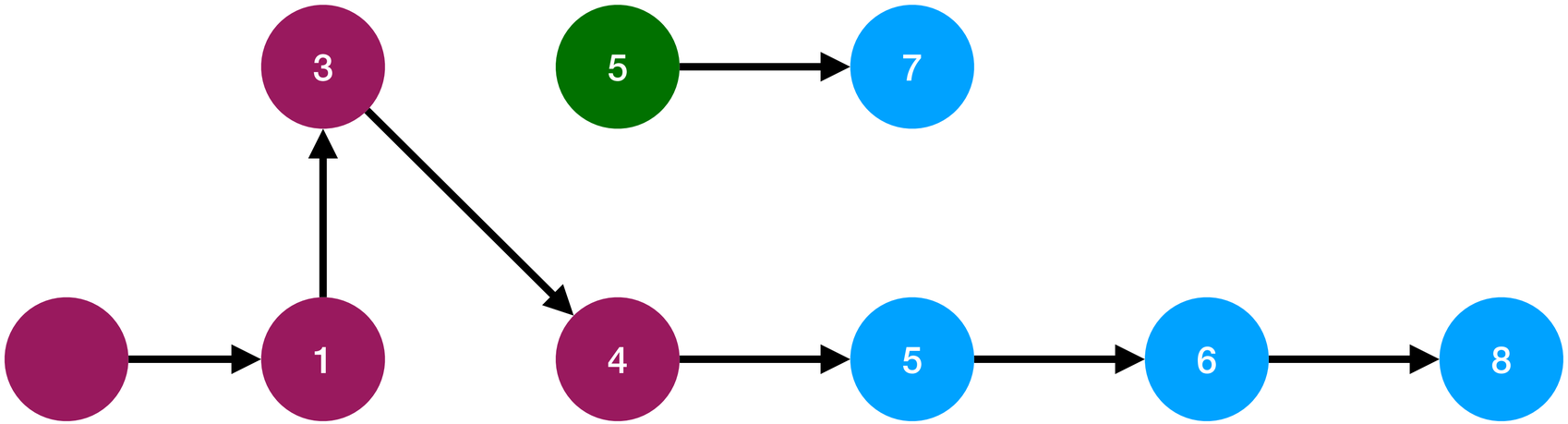
Merge of two sorted LinkedLists



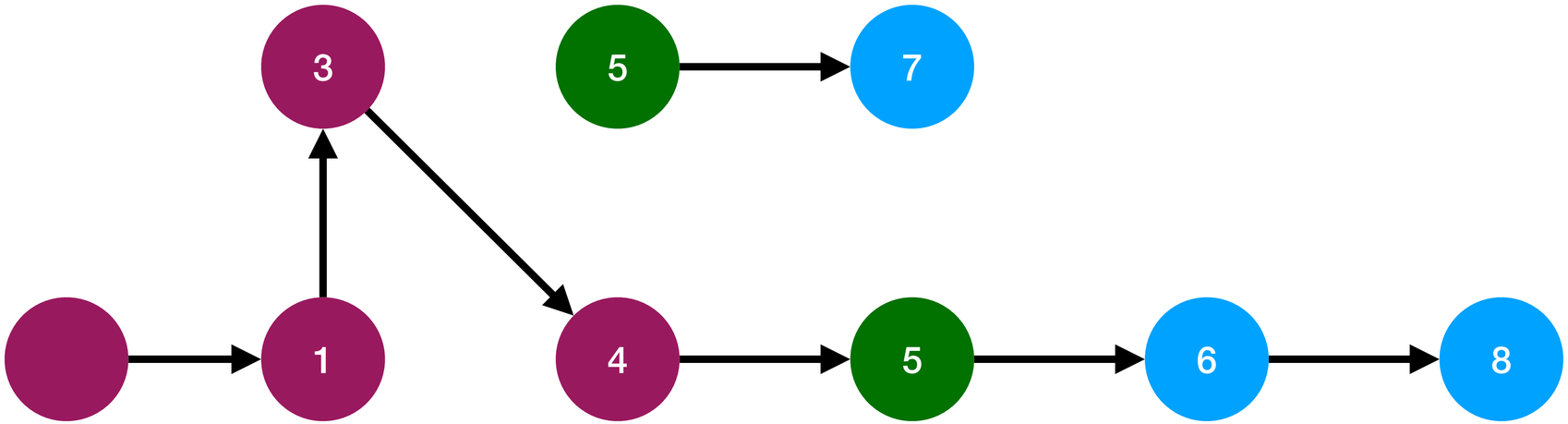
Merge of two sorted LinkedLists



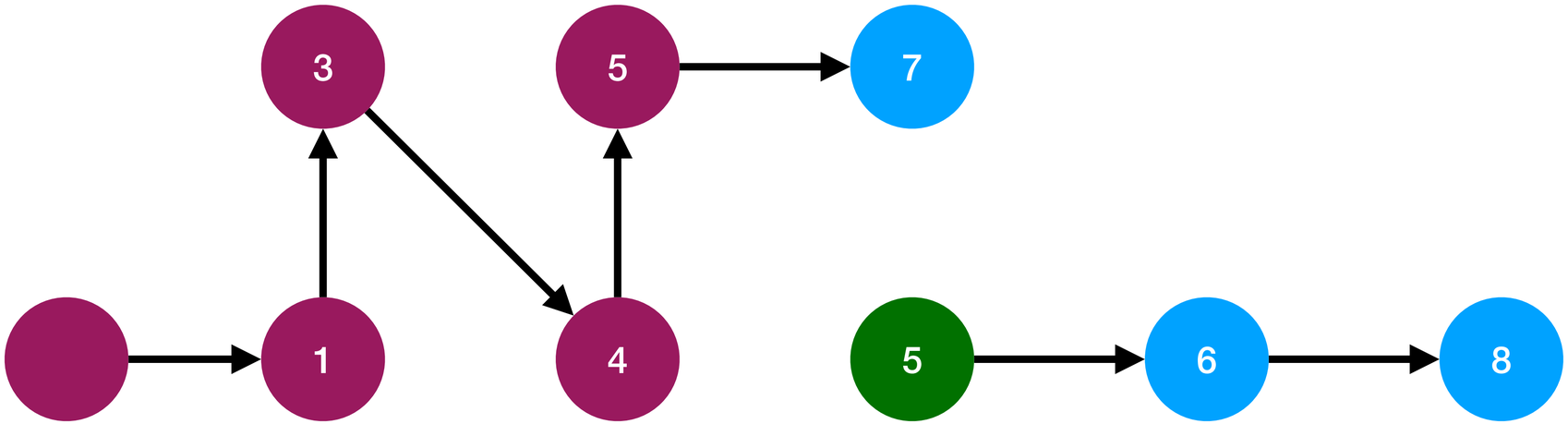
Merge of two sorted LinkedLists



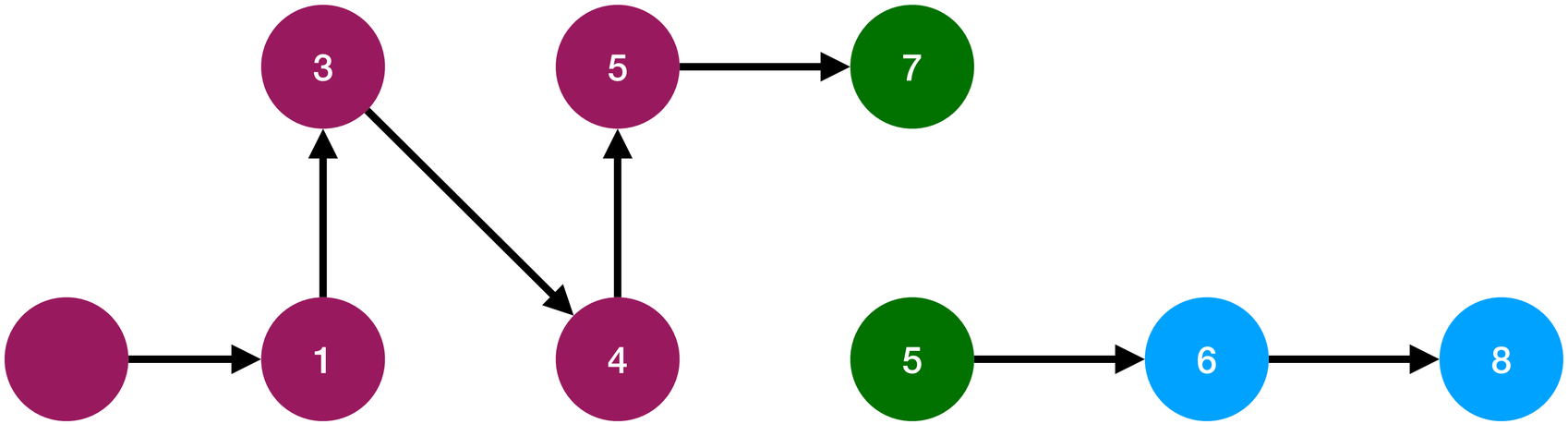
Merge of two sorted LinkedLists



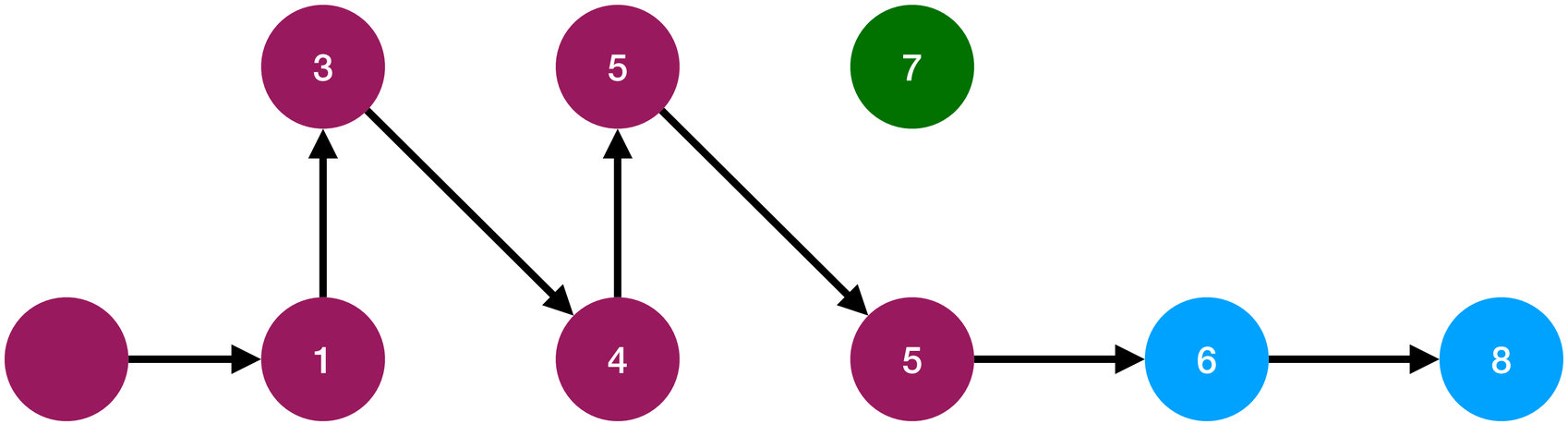
Merge of two sorted LinkedLists



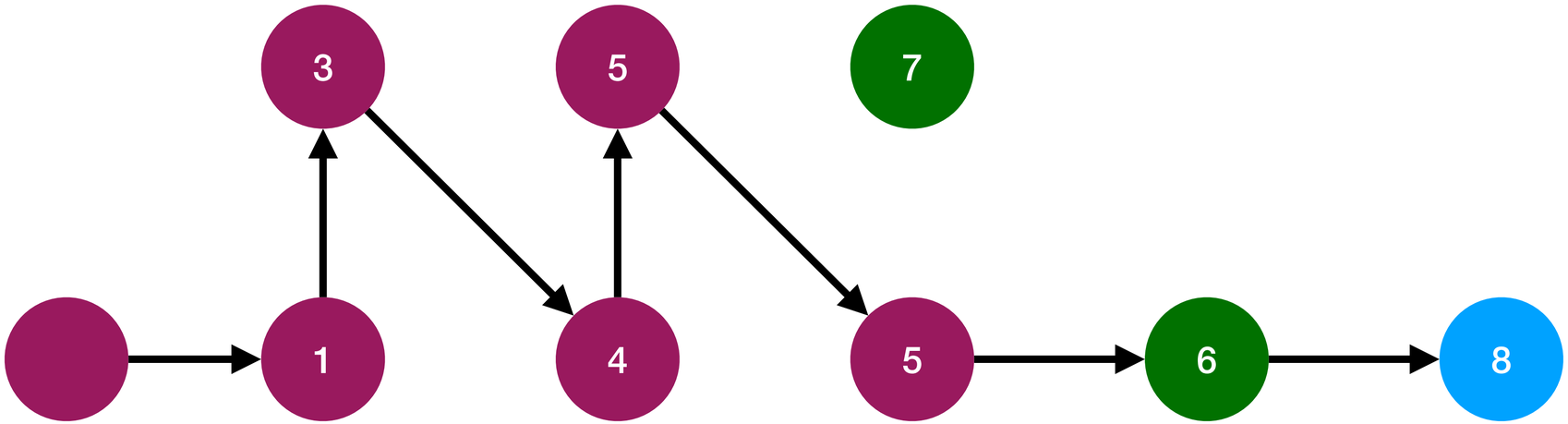
Merge of two sorted LinkedLists



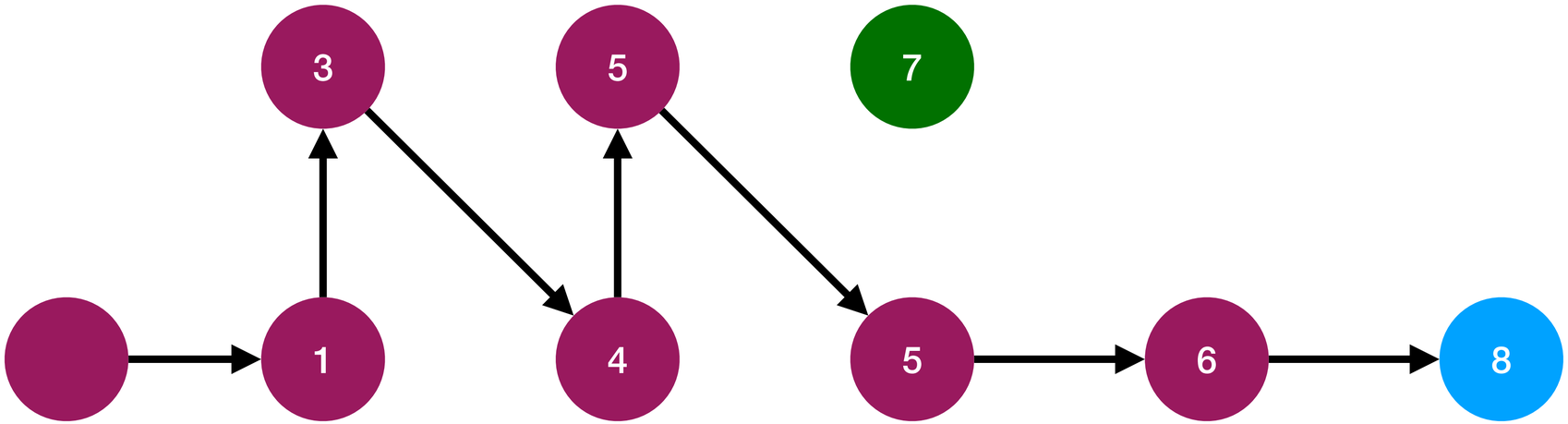
Merge of two sorted LinkedLists



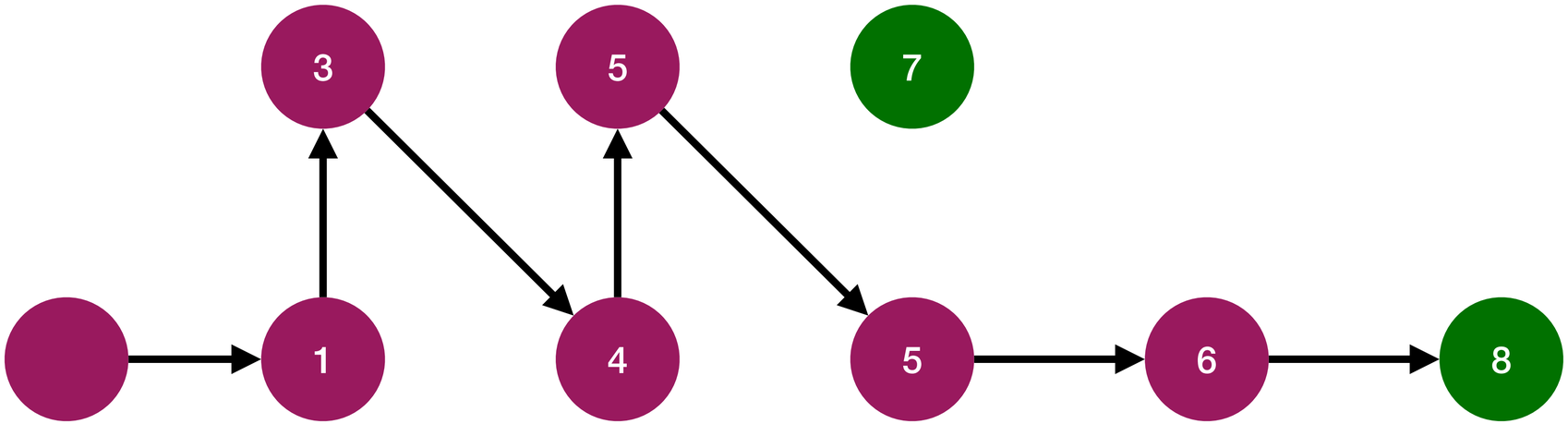
Merge of two sorted LinkedLists



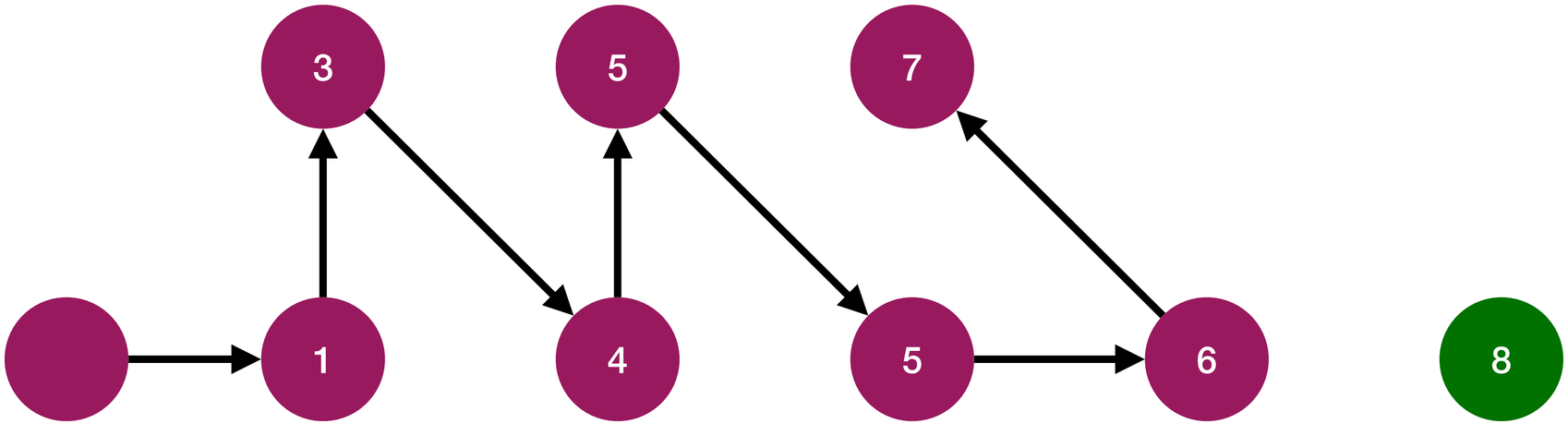
Merge of two sorted LinkedLists



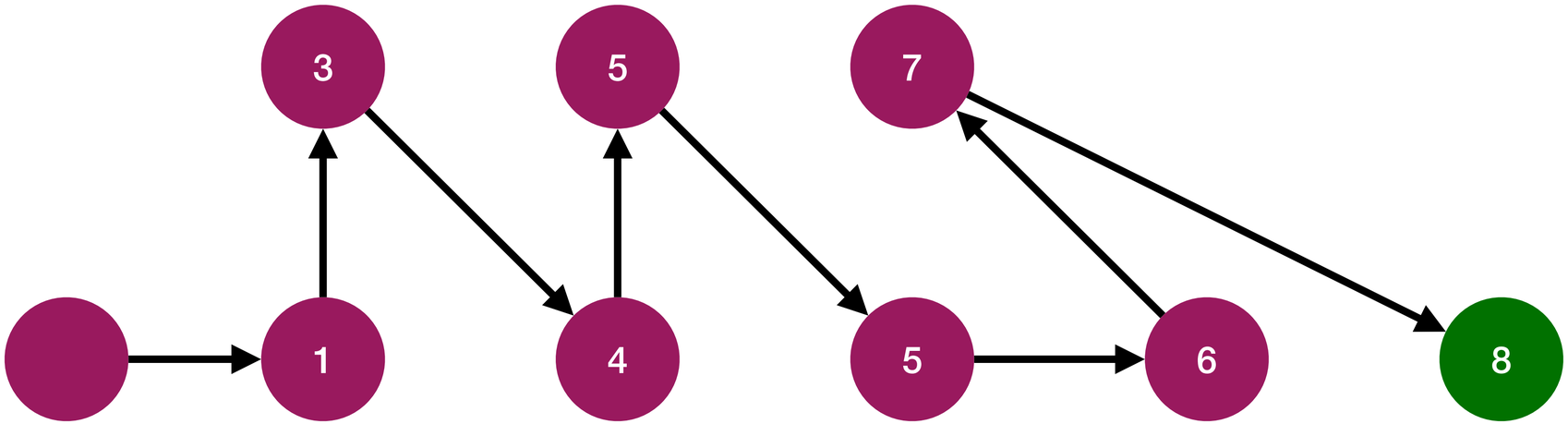
Merge of two sorted LinkedLists



Merge of two sorted LinkedLists

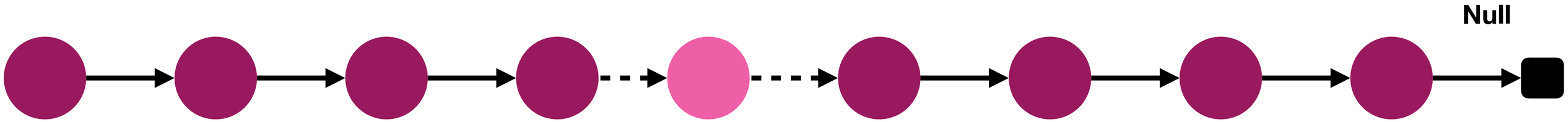


Merge of two sorted LinkedLists



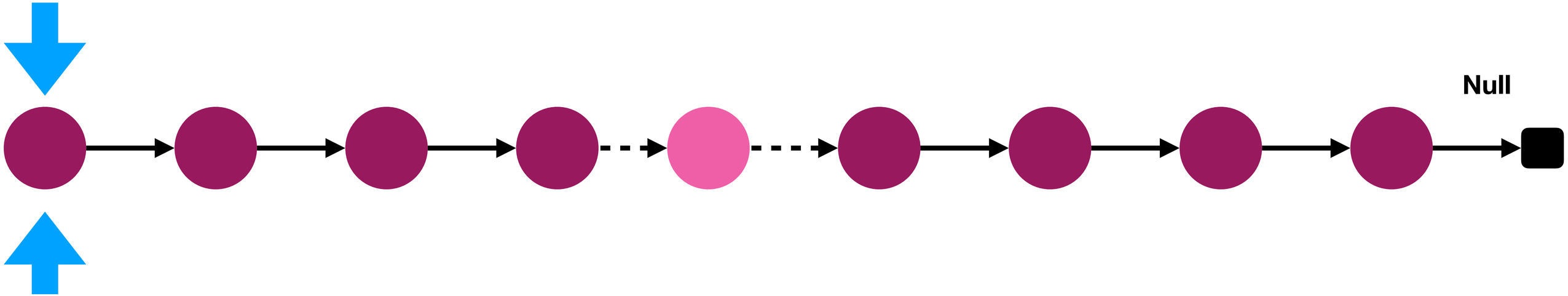
Question 2.1.5 Sorting a simply linked list, how to find the middle ?

- How to find the node in the middle in just one pass (you don’t know the length)



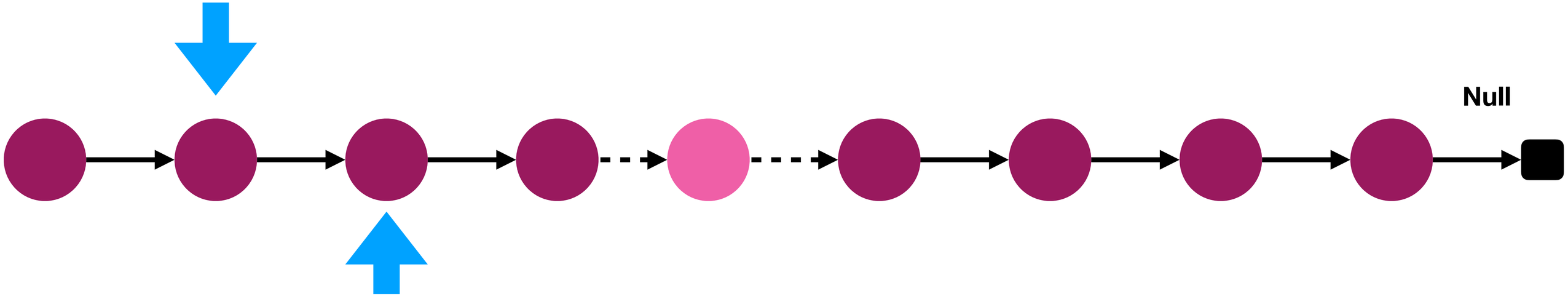
Question 2.1.5 Sorting a simply linked list, how to find the middle ?

- How to find the node in the middle in just one pass (you don’t know the length)



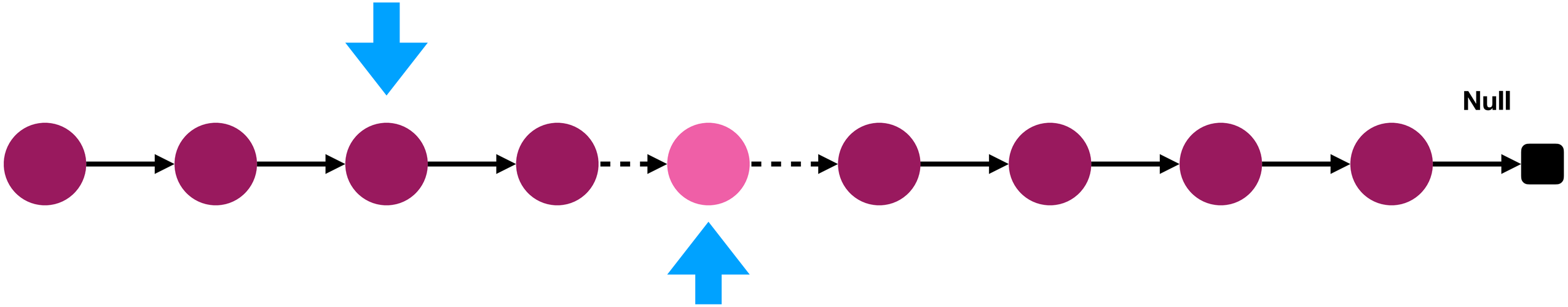
Question 2.1.5 Sorting a simply linked list, how to find the middle ?

- How to find the node in the middle in just one pass (you don’t know the length)



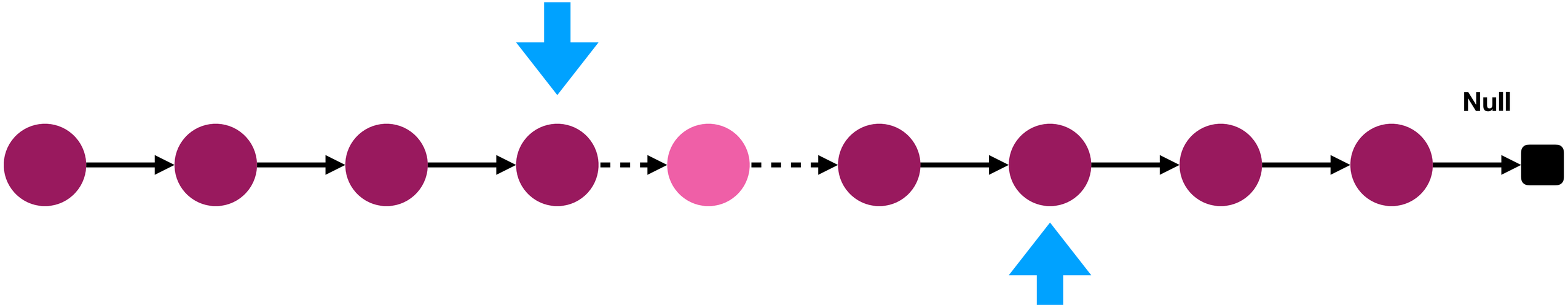
Question 2.1.5 Sorting a simply linked list, how to find the middle ?

- How to find the node in the middle in just one pass (you don't know the length)



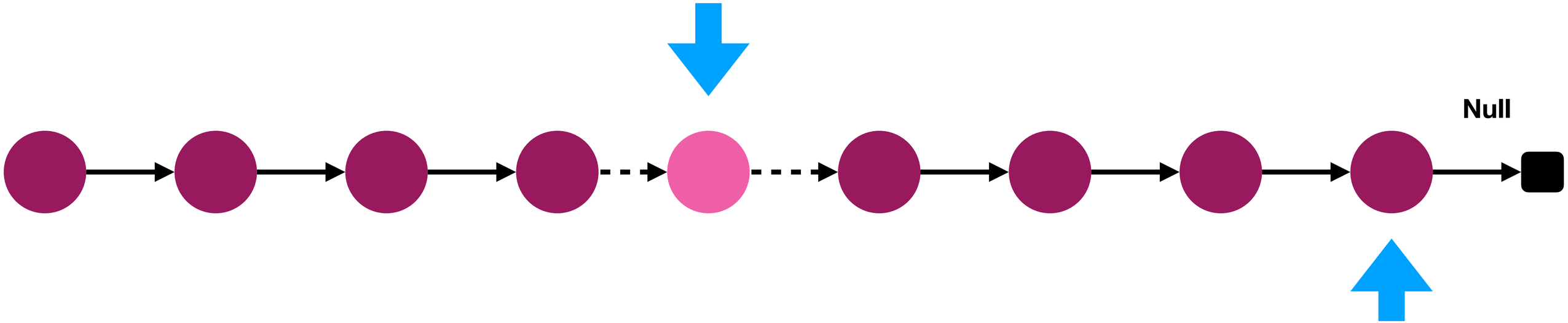
Question 2.1.5 Sorting a simply linked list, how to find the middle ?

- How to find the node in the middle in just one pass (you don't know the length)



Question 2.1.5 Sorting a simply linked list, how to find the middle ?

- How to find the node in the middle in just one pass (you don’t know the length)



Question 2.1.5 Sorting a simply linked list, how to find the middle ?

```
protected Node mergeSort(Node head) {
    if (head == null || head.next == null) {
        return head;
    }
    // find the end of the first half
    Node slow = head;
    Node fast = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    Node second = slow.next;
    slow.next = null;
    return merge(mergeSort(head), mergeSort(second));
}
```

Question 2.1.6 Number of unordered pairs

Design an efficient algorithm for counting the number of pairs of disordered values. For example in the sequence 1,3,2,5,6,4,8 there are the pairs (3,2),(5,4),(6,4) which are unordered. Justify the complexity of your algorithm and give its pseudo code.

Question 2.1.6 Number of unordered pairs

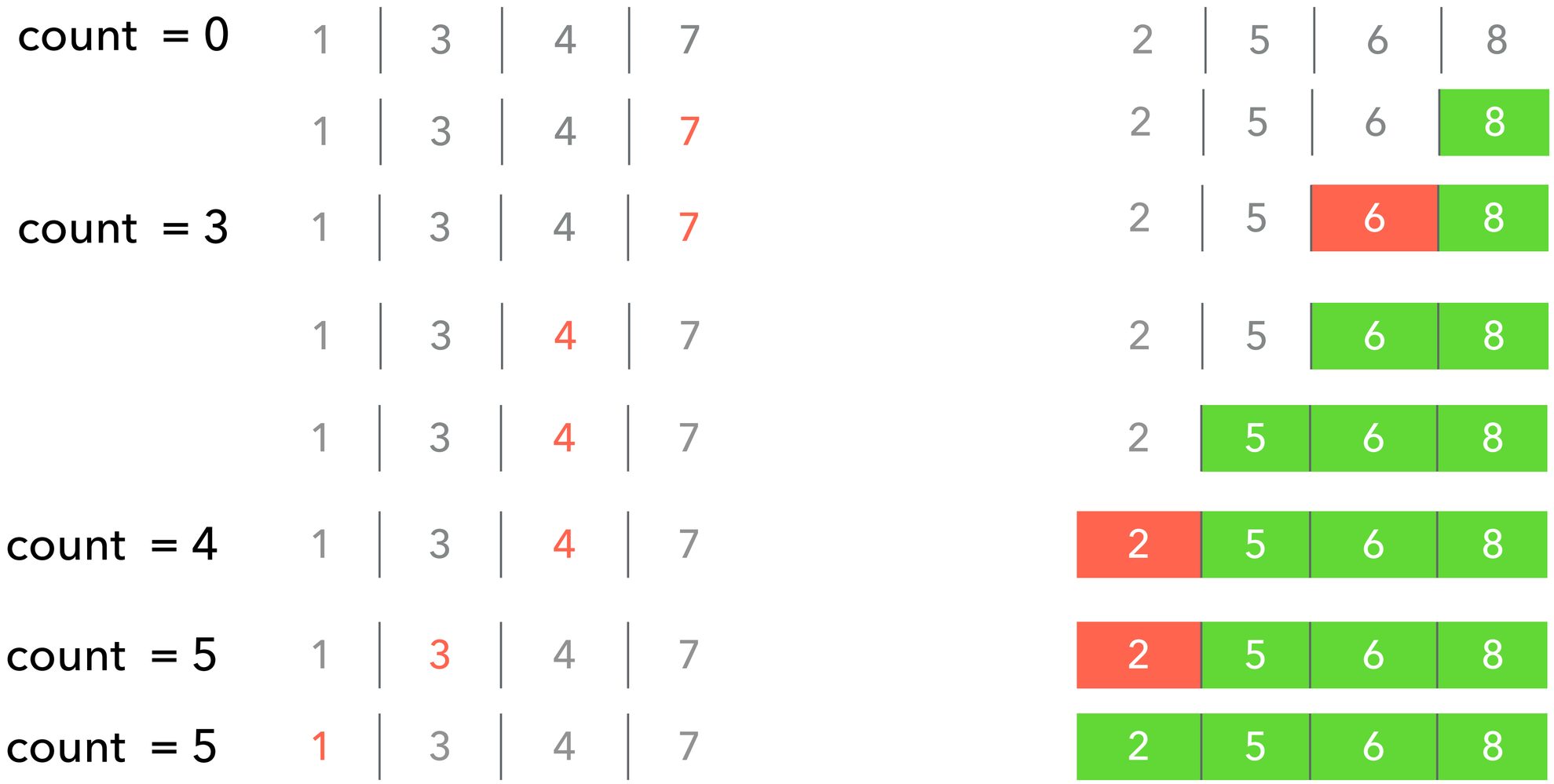
Design an efficient algorithm for counting the number of pairs of disordered values. For example in the sequence 1,3,2,5,6,4,8 there are the pairs (3,2),(5,4),(6,4) which are unordered. Justify the complexity of your algorithm and give its pseudo code.

Hint: Assume two arrays A and B, let $A \cdot B$ be the array result of the concatenation of A and B. Let $nUnsorted(A)$ be the number of unsorted pairs in an array A.
We have the following property that you can prove:

$$nUnsorted(A \cdot B) = nUnsorted(A) + nUnsorted(B) + |\{(i,j) : A[i] > B[j]\}|$$

Question 2.1.6 Number of unordered pairs

Computing the unsorted elements in A and B is linear if the two arrays are sorted



Question 2.1.6 Number of unordered pairs

Computing the unsorted elements in A and B is linear if the two arrays are sorted

```
public static int wrongOrder(int[] array, int lo, int mid, int hi) {
    int[] A = Arrays.copyOfRange(array, lo, mid + 1);
    int[] B = Arrays.copyOfRange(array, mid + 1, hi + 1);

    int posB = B.length;
    int count = 0;

    for (int i = A.length - 1; i >= 0; i--) {
        // INV: count = \sum_{k=i+1}^{A.length-1} #{j ∈ [0..B.length − 1] | B[j] < A[k]}
        while (posB > 0 && B[posB - 1] >= A[i]) {
            posB--;
        }
        count += posB;
    }
    return count;
}
```

Question 2.1.6 Number of unordered pairs

```
public static int countInversions(int[] array) {
    return numberUnsortedPairs(array, 0, array.length - 1);
}

public static int numberUnsortedPairs(int[] array, int lo, int hi) {
    if (lo >= hi) {
        return 0;
    }
    int mid = (lo + hi) / 2;
    int nA = numberUnsortedPairs(array, lo, mid);
    int nB = numberUnsortedPairs(array, mid + 1, hi);
    int wab = wrongOrder(array, lo, mid, hi);
    merge(array, lo, mid, hi);
    return nA + nB + wab;
}
```

Question 2.1.7 Comparable vs Comparator

Imagine that we want to sort a collection of *Person* objects lexicographically by their (weight, age, height) but also *Student* objects by their (age, grade, year), how to avoid duplicating the sorting algorithm specifically for these classes?

Explain why the notions of *Comparable* and *Comparator* of Java are useful for this?
Explain how you would implement an efficient Comparator for String.

Question 2.1.7 Comparable vs Comparator

- **Using Comparable**

```
public class Person {
    int age;
    String name;
    int height;

    public Person(String name, int age, int height) {
        this.name  = name;
        this.age = age;
        this.height = height;
    }

    public int getAge() {
        return this.age;
    }

    public int getHeight() {
        return this.height;
    }
}
```

```
Person[] people = new Person[] {
                            new Person("Tom", 15, 177),
                            new Person("Hannah", 16, 170),
                            new Person("Ludovic", 2, 80)
};
```

```
Arrays.sort(people);
```

```
public class Person implements Comparable<Person> {
    int age;
    String name;
    int height;

    public Person(String name, int age, int height) {
        this.name  = name;
        this.age = age;
        this.height = height;
    }

    public int getAge() {
        return this.age;
    }

    public int getHeight() {
        return this.height;
    }

    @Override
    public int compareTo(Person o) {
        return this.age - o.age;
    }
}
```

Question 2.1.7 Comparable vs Comparator

- Using Comparator

```
public class Person {
    int age;
    String name;
    int height;

    public Person(String name, int age, int height) {
        this.name  = name;
        this.age = age;
        this.height = height;
    }

    public int getAge() {
        return this.age;
    }

    public int getHeight() {
        return this.height;
    }
}
```

```
public class PersonComparator implements Comparator<Person> {

    @Override
    public int compare(Person p1, Person p2) {
        return p1.age - p2.age;
    }
}
```

```
Arrays.sort(people, new PersonComparator());
```

```
Arrays.sort(people, new Comparator<Person>() {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getAge() - p2.getAge();
    }
});
```

```
Arrays.sort(people, (p1, p2) -> p1.name.compareTo(p2.name));
```

```
Arrays.sort(people, Comparator.comparingInt(Person::getHeight));
```

Reminder LEPL1402: Function/Method Reference syntactic sugar

```java
import java.util.function.*;

public class Example {
    public static void main(String[] args) {
        // Example 1: static method reference
        IntUnaryOperator absFunc = Math::abs;
        System.out.println(absFunc.applyAsInt(-5)); // 5
        // Example 2: instance method reference
        Function<String, String> lower = String::toLowerCase;
        System.out.println(lower.apply("HELLO")); // “hello"
        // Example 2b: instance method reference with custom class
        ToIntFunction<Person> heightFunc = Person::getHeight;
        System.out.println(heightFunc.applyAsInt(new Person(175))); // 175
        // Example 3: custom object
        Person p = new Person(180);
        Supplier<Integer> heightGetter = p::getHeight;
        System.out.println(heightGetter.get()); // 180
    }
}

class Person {
    private int height;
    public Person(int height) { this.height = height; }
    public int getHeight() { return height; }
}
```

Question 2.1.8 Stable sort from an unstable one ?

Is it possible to get a stable sort starting from an unstable sorting algorithm? How?

We can encapsulate the value to be sorted in an object that contains its position, and perform a tie-break in the comparison function.

Question 2.1.9 Find the third largest value

How to find the third largest value in an array ?

Question 2.1.9 Find the third largest value

How to find the third largest value in an array ? We can use the same algorithm as to find the minimum, it is even linear !

Question 2.1.9 Find the third largest value

How to find the third largest value in an array ? We can use the same algorithm as to find the minimum, it is even linear !

```
public static int findThirdLargest(int [] array) {
    int max1 = Integer.MIN_VALUE; // largest
    int max2 = Integer.MIN_VALUE; // second largest
    int max3 = Integer.MIN_VALUE; // third largest
    for (Integer i : array) {
        if (i > max1) {
            max3 = max2; max2 = max1; max1 = i;
        } else if (i > max2) {
            max3 = max2; max2 = i;
        } else if (i > max3) {
            max3 = i;
        }
    }
    return max3;
}
```

Find k-th largest: Which one do you prefer ?

```java
/**
 * Find the k-th largest element in an array.
 * @param array
 * @param k 1-based index of the largest element to find
 * @return the k-th largest element
 */
public static int findKthLargest(int[] array, int k) {
    if (k < 1 || k > array.length) {
        throw new IllegalArgumentException("k is out of range");
    }

    int[] top = new int[k];
    // Initialize with minimum possible values
    for (int i = 0; i < k; i++) {
        top[i] = Integer.MIN_VALUE;
    }

    for (int num : array) {
        // If current number is larger than the smallest in top[]
        if (num > top[0]) {
            // Insert it in the correct place (keep array sorted ascending)
            top[0] = num;
            // Bubble up to restore order
            for (int i = 1; i < k && top[i - 1] > top[i]; i++) {
                int temp = top[i];
                top[i] = top[i - 1];
                top[i - 1] = temp;
            }
        }
    }

    // After processing all elements, top[0] = k-th largest
    return top[0];
}
```

```java
/**
 * Find the k-th largest element in an array.
 * @param array
 * @param k 1-based index of the largest element to find
 * @return the k-th largest element
 */
public static int findKthLargest(int[] array, int k) {
  Arrays.sort(array);
  return array[array.length - k];
}
```

Question 2.1.10 Median

How would you get the median of an array of values (so the n/2 th value)? What is the time complexity of your algorithm?

Question 2.1.10 Median

How would you get the median of an array of values (so the n/2 th value)? What is the time complexity of your algorithm?

We can sort the array, and then take the element at the middle index. Complexity is *O(n log(n))*, good enough. Can we do better ?

Question 2.1.10 Median

What does the partition function do?

```
public class Quick
{
   public static void sort(Comparable[] a)
   {
      StdRandom.shuffle(a);            // Eliminate dependence on input.
      sort(a, 0, a.length - 1);
   }

   private static void sort(Comparable[] a, int lo, int hi)
   {
      if (hi <= lo) return;
      int j = partition(a, lo, hi);  // Partition (see page 291).
      sort(a, lo, j-1);              // Sort left part a[lo .. j-1].
      sort(a, j+1, hi);              // Sort right part a[j+1 .. hi].
   }
}
```

Question 2.1.10 Median

What does the partition function do?

	i	j						
Initial values	0	6	10	1	12	8	4	7	13
Scan left, scan right	2	5	10	1	12	8	4	7	13
Exchange	2	5	10	1	7	8	4	12	13
Scan left, scan right	4	4	10	1	7	8	4	12	13
Final Exchange			4	1	7	8	10	12	13

Question 2.1.10 Median

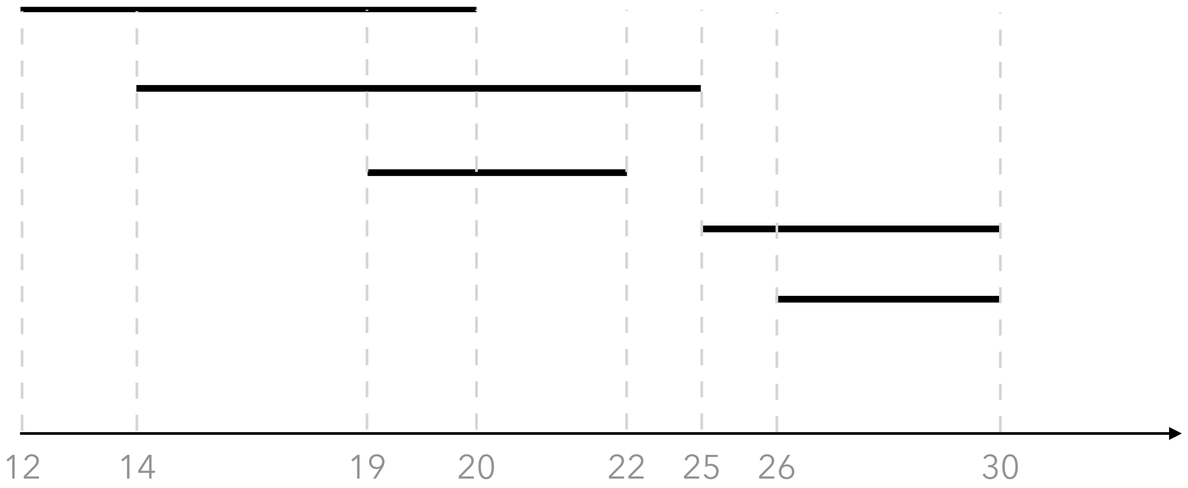
```
public static int median(int[] array) {
    int lo = 0;
    int hi = array.length;
    int i = lo;
    while (i != array.length / 2) {
        int i = partition(array, lo, hi);
        if i < array.length / 2 {
            lo = i;
        } else if i > array.length / 2 {
            hi = i;
        }
    }
}
```

Question 2.1.11 AUTOBOXING, UNBOXING

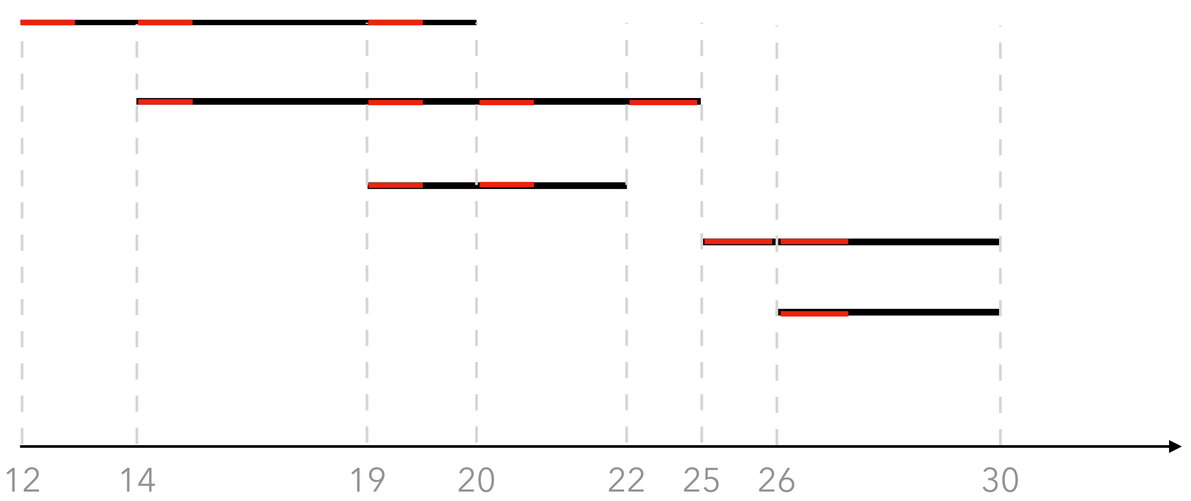
What is Autoboxing and Unboxing in Java? How can this impact the performance of a sorting algorithm?

Autoboxing in Java refers to the automatic conversion of primitive types (like int, char, etc.) into their corresponding wrapper classes (like Integer, Character, etc.) when an object is required, while **unboxing** is the reverse process where the wrapper class is converted back into its primitive type

Question 2.1.15 Training Sessions



Question 2.1.15 Training Sessions



Room = 3

- Go from min start time to max time among sessions
- At each time step scan all sessions to check if overlap
- Max overlap is the number of room needed at the same time

Question 2.1.15 Training Sessions: Brute force solution

```
public static int minFacilitiesRequiredBruteForce(int[][] sessions) {
    int minTime = Integer.MAX_VALUE;
    int maxTime = Integer.MIN_VALUE;

    for (int[] session : sessions) {
        minTime = Math.min(minTime, session[0]);
        maxTime = Math.max(maxTime, session[1]);
    }

    int maxOverlap = 0;

    for (int time = minTime; time <= maxTime; time++) {
        int overlap = 0;

        for (int[] session : sessions) {
            if (session[0] <= time && session[1] > time) {
                overlap++;
            }
        }
        maxOverlap = Math.max(maxOverlap, overlap);
    }

    return maxOverlap;
}
```

Question 2.1.15 Training Sessions: Polynomial but not optimal

```java
public int minFacilitiesRequiredLessBruteForce(int[][] sessions) {
    int n = sessions.length;
    int maxOverlap = 0;

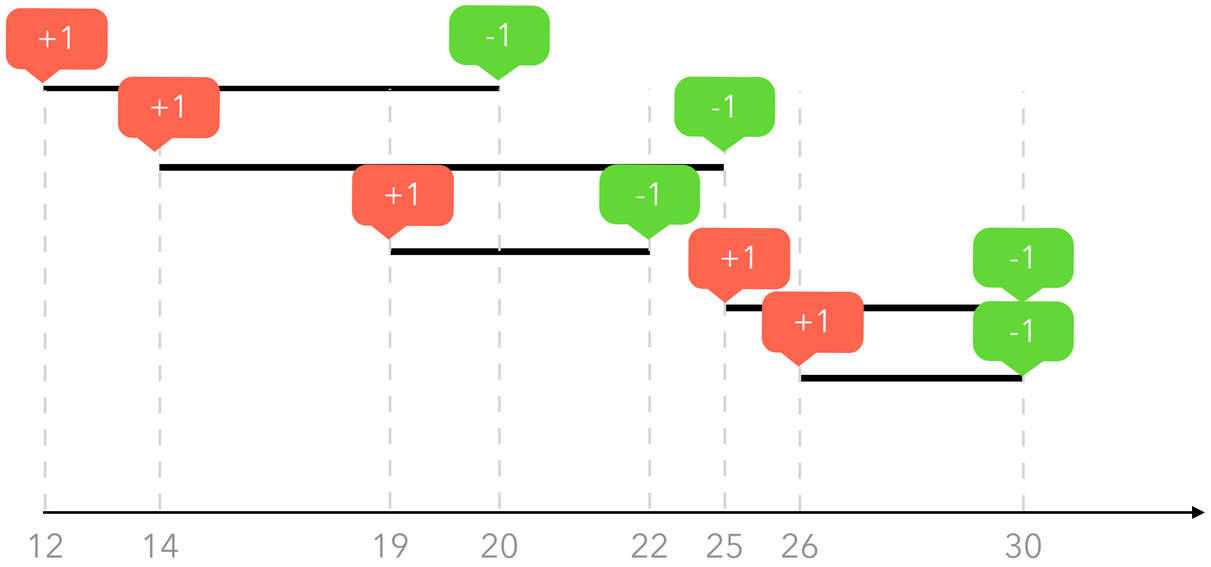
    for (int i = 0; i < n; i++) {
        int t = sessions[i][0];          // check at this start time
        int count = 0;

        for (int j = 0; j < n; j++) {
            // Count intervals that cover time t (half-open: start <= t < end)
            if (sessions[j][0] <= t && t < sessions[j][1]) {
                count++;
            }
        }

        if (count > maxOverlap) maxOverlap = count;
    }

    return maxOverlap;
}
```

Question 2.1.15 Training Sessions: Sweep Line





Question 2.1.15 Training Sessions

```java
public int minFacilitiesRequired(int[][] sessions) {
    // sweep line algorithm
    List<int[]> events = new ArrayList<>();
    for (int[] session : sessions) {
        events.add(new int[]{session[0], 1});
        events.add(new int[]{session[1], -1});
    }
    events.sort((a, b) -> (a[0] == b[0]) ? a[1] - b[1] : a[0] - b[0]);
    int max = 0;
    int current = 0;
    for (int[] event : events) {
        current += event[1];
        max = Math.max(max, current);
    }
    return max;
}
```